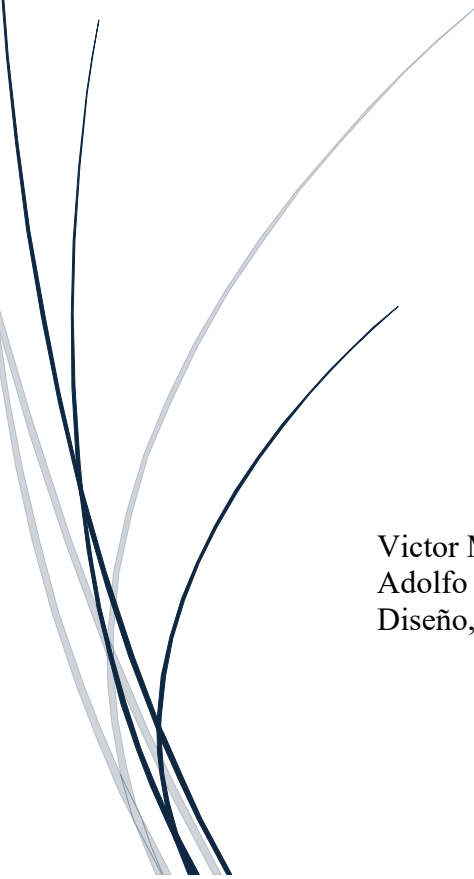




4-9-2026

Multiple Mode Register

T4



Victor Manuel Sandoval Maciel
Adolfo Meza Ibarra
Diseño, verificación y validación de sistemas digitales

Content

STATEMENT.	2
DEVELOPMENT.....	3
1. PIPO.	3
Test bench PIPO	3
2. SISO MSB.....	3
Test bench SISO MSB.....	3
3. SISO LSB.	4
Test bench SISO LSB.....	4
4. SIPO MSB	4
Test bench SIPO MSB.....	4
5. PISO LSB CIRCULAR	5
Test bench PISO LSB CIRCULAR	5
6. PISO MSB SIGN.....	5
Test bench PISO MSB SIGN	5
7. Load and Shift (to the left circular)	6
Test bench Load and shift to the left circular	6
8. SIPO LSB first.....	6
Test bench SIPO LSB first	6
Código.	7
MultipleModeRegister - [Top].	7
Poli_Register – [Modulo].	7
Poli_Register_Modes_pkg – [Package].	8
MultipleModeRegister_tb – [TestBench].	9

STATEMENT.

This is a collaborative task. Based on learnings of register types and design in SystemVerilog create a multiple mode register that can work as 8 of these modes (whichever you prefer):

1. PIPO.
2. SISO (MSB first).
3. SISO (LSB first).
4. PISO (MSB first) (inserting 0s).
5. PISO (MSB first) (repeating MSB / sign conservation).
6. PISO (MSB first) (circular).
7. PISO (LSB first) (inserting 0s).
8. PISO (LSB first) (circular).
9. SIPO (MSB first).
10. SIPO (LSB first).
11. Load and Shift (to the right) (inserting 0s).
12. Load and Shift (to the right) (repeating MSB / sign conservation).
13. Load and Shift (to the left) (inserting 0s).
14. Load and Shift (to the right) (circular).
15. Load and Shift (to the left) (circular).

Restrictions:

- Module must be parametrized on data width (DW) of the register.
- Register must include an enable and sync_reset signals.
- Module must include another parameter that works as a selector of type of register to synthesize. Add a package file to include macros which give nice names for each of the selector values.
- Inputs Outputs can be chosen as designer preference and it must be documented in the code.
- The maximum number of flip-flops to use is DW (DW being the size of the register).

Generate also a testbench for each type of the register types (8) where module is instantiated and tested in that mode.

Deliverables:

- Source file of multiple mode register implementation (1).
- Testbench files (1/8) for each register mode
- Pictures (8) of waveforms showing the results for each execution.

Hint:

- Use combinational always blocks for each mode process and use a single flip flop always block to generate DW flops.

Extra points (research):

- Make the test benches (8) to auto-check that register is working fine. (20% extra)
- Make a single test bench (1) that selects the parameter and test scenarios based on a plus arg. (40% extra but previous bullet must also be fulfilled)

DEVELOPMENT.

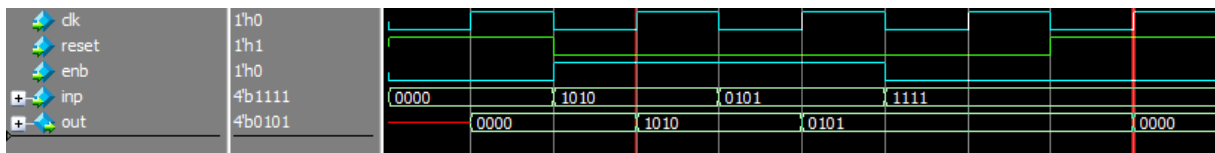
1. PIPO.

Comportamiento PIPO (Parallel Input Parallel Output)

Su comportamiento es simple:

- La entrada completa cuyo tamaño es DW (Parametro para definir el tamaño) bits, entra al mismo tiempo.
- En el flanco positivo de clk, si enb = 1 (enable), el registro guarda inp (Input).
- La salida out muestra el valor almacenado .
- Si enb = 0, conserva el valor anterior.
- Si reset = 1, en el siguiente posedge clk el registro se limpia a 0, porque tu reset es síncrono

Test bench PIPO



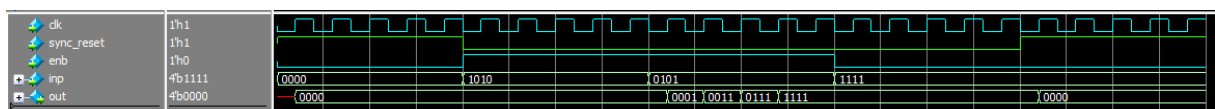
2. SISO MSB.

Comportamiento SISO (Single Input Single Output y Most Significant Bit).

Su comportamiento se define como:

- La entrada es de a un bit en uno, cuyo tamaño interior se ve definido por DW (Parametro para definir el tamaño) bits, entra al mismo tiempo.
- Conforme entra el primer bit del bus (3'b1010, entra 0 de la posición [0]) y se almacena como MSB, ósea comenzando por el bit [0] del arreglo de tamaño DW.
- En el flanco positivo de clk, si enb = 1 (enable), el registro guarda inp (Input).
- La salida out muestra el valor almacenado .
- Si enb = 0, conserva el valor anterior.
- Si reset = 1, en el siguiente posedge clk el registro se limpia a 0, porque tu reset es síncrono.

Test bench SISO MSB



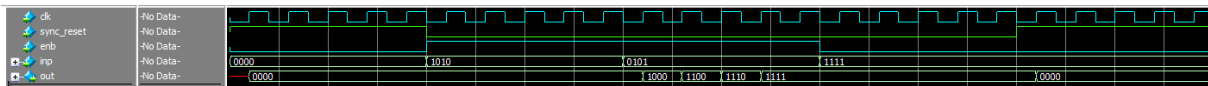
3. SISO LSB.

Comportamiento SISO (Single Input Single Output y Less Significant Bit).

Su comportamiento se define como:

- La entrada es de a un bit en uno, cuyo tamaño interior se ve definido por DW (Parámetro para definir el tamaño) bits, entra al mismo tiempo.
- Conforme entra el primer bit del bus (3'b1010, entra 0 de la posición [0]) y se almacena como LSB, ósea comenzando por el bit [DW-1] del arreglo de tamaño DW.
- En el flanco positivo de clk, si enb = 1 (enable), el registro guarda inp (Input).

Test bench SISO LSB



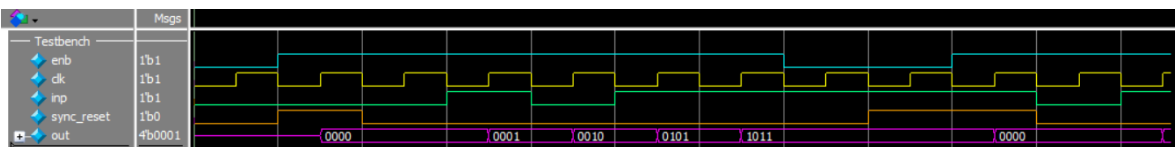
4. SIPO MSB

Comportamiento SIPO (Serial Input Parallel Output y Most Significant Bit).

Su comportamiento se define como:

- La entrada es de a un bit a la vez, que se toma de inp[0]. El tamaño del registro se define con DW.
- En cada flanco positivo de clk, si enb = 1 (enable), el registro se recorre a la izquierda y el bit nuevo entra por la posición [0].
- El primer bit que se envía termina en el MSB después de DW ciclos, por eso se llama MSB first. Si se envían los bits 1, 0, 1, 1 el registro pasa por 0001, 0010, 0101 y 1011.
- La salida out muestra los DW bits en paralelo. El dato está completo hasta el ciclo DW.
- Este modo no usa load_shift, porque nunca carga en paralelo: solo desplaza.
- Si enb = 0, conserva el valor anterior.
- Si sync_reset = 1, en el siguiente posedge clk el registro se limpia a 0, porque el reset es síncrono.

Test bench SIPO MSB

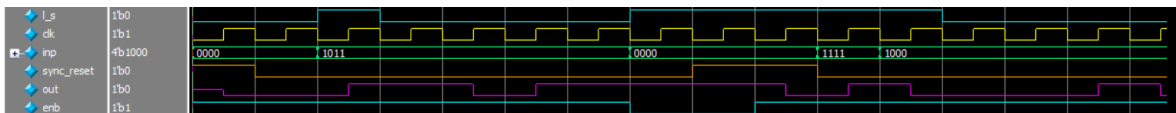


5. PISO LSB CIRCULAR

Comportamiento PISO (Parallel Input Serial Output), Less Significant Bit first y circular. Su comportamiento se define como:

- La entrada completa de DW bits entra al mismo tiempo cuando $l_s = 1$ (load or shift), en un solo flanco positivo de clk .
- Cuando $l_s = 0$, el registro rota a la derecha: el bit que sale por el LSB vuelve a entrar por el MSB, así que ningún bit se pierde.
- La salida serie es el LSB del registro, y entrega el dato empezando por el bit menos significativo. Con el dato 1011 la secuencia que sale es 1, 1, 0, 1.
- Al ser circular, después de DW rotaciones el registro recupera el dato original (1011, 1101, 1110, 0111, 1011), y la transmisión se puede repetir sin volver a cargar.
- Si $enb = 0$, conserva el valor anterior.
- Si $sync_reset = 1$, en el siguiente posedge clk el registro se limpia a 0, porque el reset es síncrono.

Test bench PISO LSB CIRCULAR



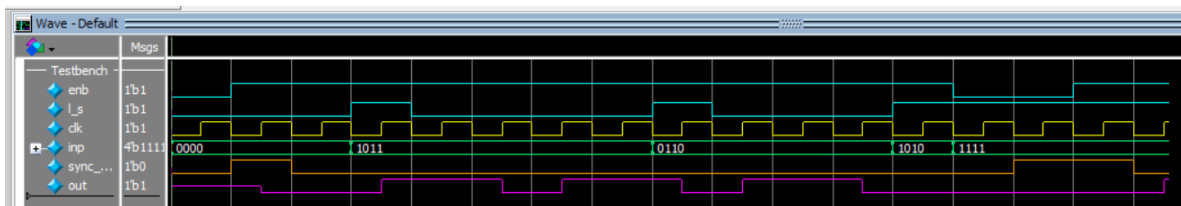
6. PISO MSB SIGN

Comportamiento PISO (Parallel Input Serial Output) con conservación de signo.

Su comportamiento se define como:

- La entrada completa de DW bits entra al mismo tiempo cuando $l_s = 1$ (load or shift), en un solo flanco positivo de clk .
- Cuando $l_s = 0$, el registro desplaza a la derecha replicando el MSB, es decir, el bit de signo se copia sobre sí mismo en cada desplazamiento. Esto es un corrimiento aritmético.
- La salida serie es el LSB del registro, y entrega el dato empezando por el bit menos significativo.
- Como el MSB nunca cambia, el signo se conserva durante toda la transmisión: un dato negativo satura en todo unos (1011, 1101, 1110, 1111) y uno positivo satura en todo ceros (0110, 0011, 0001, 0000).
- Si $enb = 0$, conserva el valor anterior.
- Si $sync_reset = 1$, en el siguiente posedge clk el registro se limpia a 0, porque el reset es síncrono.

Test bench PISO MSB SIGN



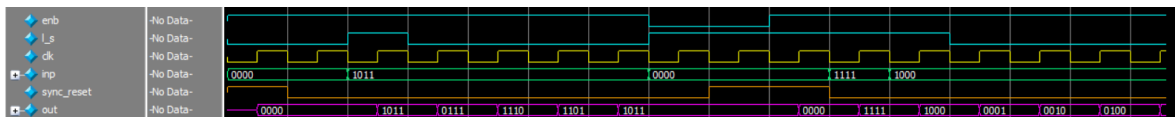
7. Load and Shift (to the left circular)

Comportamiento Load and Shift a la izquierda, de forma circular.

Su comportamiento se define como:

- La entrada completa de DW bits entra al mismo tiempo cuando $l_s = 1$ (load or shift), en un solo flanco positivo de clk .
- Cuando $l_s = 0$, el registro rota a la izquierda: el MSB sale y vuelve a entrar por el LSB, así que ningún bit se pierde.
- La salida out es paralela y muestra los DW bits, de modo que el contenido rotando se observa directamente (1011, 0111, 1110, 1101, 1011).
- Con un solo bit en alto se aprecia el recorrido completo: 1000, 0001, 0010, 0100, 1000. El salto de 1000 a 0001 se debe a que el MSB reentra por el extremo opuesto.
- Al ser circular, después de DW rotaciones el registro recupera el dato original.
- Si $enb = 0$, conserva el valor anterior.
- Si $sync_reset = 1$, en el siguiente posedge clk el registro se limpia a 0, porque el reset es síncrono.

Test bench Load and shift to the left circular



8. SIPO LSB first.

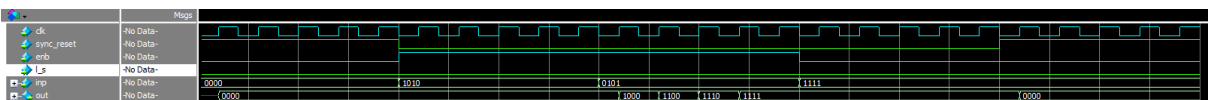
Comportamiento SIPO LSB First (Serial Input Parallel Output y Least Significant Bit First).

Su comportamiento se define como:

- La entrada se realiza de un bit a la vez, mientras que el registro interno tiene un tamaño definido por DW.
- En cada ciclo de reloj entra un nuevo bit serial mediante $inp[0]$.
- Al ser LSB First, el bit de menor peso $reg_r[0]$ es el primero en desplazarse hacia la salida. Por ello, cada nuevo bit entra por la posición MSB, es decir, $reg_r[DW-1]$.
- En cada flanco positivo de clk , si $enb = 1$ (enable), los bits almacenados se desplazan una posición hacia el LSB y el nuevo bit $inp[0]$ se almacena en el MSB.

Después de DW ciclos, los bits recibidos pueden leerse en paralelo mediante $out[DW-1:0]$.

Test bench SIPO LSB first



Código.

MultipleModeRegister - [Top].

```
import Poli_Register_Modes_pkg::*;

module MultipleModeRegister #( parameter int DW = 4 )(
    input logic          clk,
    input logic          sync_reset,
    input logic          enb,
    input logic          l_s,
    input logic [ DW-1 :0 ] inp,
    output logic [ DW-1 :0 ] out
);

    Poli_Register #(
        .DW(DW),
        .MODE(SIPO_LSB_MODE)
    ) SipoLSB (

        .clk    (clk),
        .sync_reset (sync_reset),
        .enb    (enb),
        .l_s    (l_s),
        .inp    (inp),
        .out    (out)
    );

endmodule
```

Poli_Register – [Modulo].

```
module Poli_Register #( parameter DW= 4 )(
    input logic          clk,
    input logic          reset,
    input logic          enb,
    input logic [ DW-1 :0 ] input,
    output logic [ DW-1 :0 ] output
);

    logic [DW-1:0] reg_r;

    always_ff @(posedge clk) begin : register_label
        if(reset)
            reg_r <= '0;
        else if(enb)
```



```

        reg_r <= inp;

end:register_label

assign output = reg_r;
assign output = reg_r[DW-1:0];
endmodule

module Poli_Register #( parameter DW= 4 )(
    input  logic          clk,
    input  logic          reset,
    input  logic          enb,
    input  logic [ DW-1 :0 ]  input,
    output logic [ DW-1 :0 ]  output
);

logic [DW-1:0] reg_r;

always_ff @(posedge clk) begin : register_label
    if(reset)
        reg_r <= '0;
    else if(enb)
        reg_r <= [reg_r[DW-2:0],inp];

end:register_label

assign output = reg_r[DW-1:0];
endmodule

```

Poli_Register_Modes_pkg – [Package].

```

package Poli_Register_Modes_pkg;

    parameter int PIPO_MODE      = 0;
    parameter int SISO_MSB_MODE = 1;
    parameter int SISO_LSB_MODE = 2;

    parameter int SIPO_MSB_MODE      = 3;
    parameter int PISO_MSB_SIGN_MODE = 4;
    parameter int PISO_LSB_CIRC_MODE = 5;
    parameter int LOAD_SHIFT_LEFT_CIRC_MODE = 6;

    parameter int SIPO_LSB_MODE = 7;
endpackage

```

MultipleModeRegister_tb – [TestBench].

```
`timescale 1ns / 1ps
module MultipleModeRegister_tb;

    parameter int DW = 4;

    logic        clk;
    logic        sync_reset;
    logic        enb;
    logic        l_s;
    logic [DW-1:0] inp;
    logic [DW-1:0] out;

    // Instancia del DUT
    MultipleModeRegister #( .DW(DW) ) dut (
        .clk      (clk),
        .sync_reset (sync_reset),
        .enb      (enb),
        .l_s      (l_s),
        .inp      (inp),
        .out      (out)
    );

    // Clock
    always #2 clk = ~clk;

    // Estímulos
    initial begin
        clk      = 0 ;
        sync_reset = 1 ;
        enb      = 0 ;
        l_s      = 0 ; // Unicamente se usa para PISO y Load_shift
        inp      = '0; // Todos los bits empiezan en 0.

        // PIP0

        // reset off & enb On : Cargar 1010 y out = 1010
        #20; sync_reset = 0; enb = 1; inp = 4'b1010;

        // Cargar 0101
        #20; inp = 4'b0101;

        // Deshabilitar: out debe conservar 0101
        #20; enb = 0; inp = 4'b1111;

        // Hacer reset
        #20; sync_reset = 1;
        #20;

        $stop;
    end
endmodule
```

Para cada ejemplo se consigue que el total de registros no pase de la cantidad definida por DW.

Poli_Register sv	
Poli_Register_Modes_pkg sv	
MultipleModeRegister_tb sv	
Compilation Report - MultipleModeRegister	
Flow Summary	
<<Filter>>	
Flow Status	Successful - Sat Sep 5 18:10:48 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	MultipleModeRegister
Top-level Entity Name	MultipleModeRegister
Family	Cyclone V
Device	5CSXFC6D6F31C6
Timing Models	Final
Logic utilization (in ALMs)	N/A
Total registers	4
Total pins	11
Total virtual pins	0
Total block memory bits	0
Total DSP Blocks	0
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0
Total DLLs	0